

 声明

本课程资料仅限哈尔滨工业大学（深圳）《计算机设计与实践》2026夏季课程使用，严禁扩散或用作其他用途。

课程概况

本实验文档为哈尔滨工业大学（深圳）《计算机设计与实践》课程实验指导材料。页面顶端为实验的指导书，页面左侧为指导书的各个小节目录，右侧为小节内的索引，页面右上角可详细搜索，页面下方可切换上下小节。请务必按顺序阅读指导书，有问题积极在群内提出。

 提示

在开始之前，请回顾《数字逻辑设计》、《计算机组成原理》课程的相关理论和实践的知识、技能。

相关链接

- [点击下载课程材料](#)
- 在线答疑文档：

计算机设计与实践2026-在线...

大纲

计算机设计与实践2026-在线答疑

计算机设计与实践 2026-在线答疑

一、单周期CPU设计

提问时，请首先点击文档左侧大纲，在对应的分类下提问。

二、流水线CPU设计

提问时，请首先点击文档左侧大纲，在对应的分类下提问。

三、SoC设计

提问时，请首先点击文档左侧大纲，在对应的分类下提问。

一、单周期 CPU 设计

Q1:

ANS:

Q2:

ANS:

Q3:

ANS:

98 个字

92%

课程任务说明

本课程的项目内容采用2人小组的形式协作完成，最终目标是实现能够在FPGA开发板上运行CoreMark或LLAMA2推理程序的流水线SoC。

同学们可自行组队，找不到队友的由授课教师随机分组，**非特殊情况不得跨课表组队**。

组内成员的任务分配参考下表所示。

小组合作任务分配表

项目内容	小组任务1	小组任务2

实验1： 单周期CPU	A组指令的数据通路表、控制信号表		B组指令的数据通路表、控制信号表	
	【合作】讨论交流，共同完成完整单周期CPU的数据通路图绘制			
	A组指令的单周期CPU实现		B组指令的单周期CPU实现	
	A组指令的单周期CPU通过Basic Trace测试		B组指令的单周期CPU通过Basic Trace测试	
	【合作】讨论交流，共同把代码整合成完整的单周期CPU，并通过Basic Trace测试			
实验2： 流水线CPU及SoC	把完整的单周期CPU改造成理想流水线		把ICache、DCache集成到完整的单周期CPU	
	流水线冒险检测		为单周期CPU实现AXI总线控制器	
	暂停法解决流水线冒险		完成C语言的接口编程、实现I/O接口电路	
	前递法解决流水线冒险		使用C程序对单周期SoC进行下板测试	
	【合作】讨论交流，共同完成流水线SoC的代码整合、AXI Trace测试、下板测试、报告等			

组内成员如果在实验1负责小组任务1，则实验2时也可选择小组任务2，反之亦然。

需在课上现场检查的内容：（1）完整单周期CPU的数据通路图；（2）SoC运行CoreMark或LLAMA2推理程序的下板测试。**现场验收时，小组成员必须同时到场。**

关于Trace测试的说明：后续提交代码后，由程序自动评测给出分数，故不需在课上检查。

各班级课表

- 少部分同学若课表存在冲突，请自行根据以下课表挑选合适的时间上课。
- 换时间上课请提前告知相关任课老师，否则可能被误当成缺课。
- 各班人数均接近实验室最大容量，非必要请勿换班上课！ 自习的请自觉在其他班上课前离开！ 否则座位可能不够！

批次	周次	星期	节次	实验室	授课教师
A班	1	1	第三节-第四节	T2210	江仲鸣
	1	2	第七节-第八节		
	1	3	第一节-第四节		
	1	4	第五节-第八节		
	2	1	第一节-第四节		
	2	2	第五节-第八节		
	2	4	第一节-第四节		
	3-4	1	第五节-第八节		
	3-4	3	第一节-第四节		
	3-4	4	第五节-第八节		

批次	周次	星期	节次	实验室	授课教师
B班	1	1	第一节-第二节	T2507	江仲鸣
	1	2	第一节-第二节		
	1	4	第一节-第四节		
	1	5	第五节-第八节		
	2	1	第五节-第八节		
	2	3	第一节-第四节		
	2	4	第五节-第八节		
	3-4	1	第一节-第四节		
	3-4	2	第五节-第八节		
	3	4	第一节-第四节		
	4	5	第五节-第八节		

批次	周次	星期	节次	实验室	授课教师
C班	1	1	第三节-第四节	T2507	郑海刚
	1	2	第三节-第四节		
	1	3	第一节-第四节		
	1	4	第五节-第八节		
	2	1	第一节-第四节		
	2	2	第五节-第八节		
	2	4	第一节-第四节		
	3-4	1	第五节-第八节		
	3-4	3	第一节-第四节		
	3-4	4	第五节-第八节		

批次	周次	星期	节次	实验室	授课教师
D班	1	1	第一节-第二节	T2210	郑海刚
	1	2	第一节-第二节		
	1	4	第一节-第四节		
	1	5	第五节-第八节		
	2	1	第五节-第八节		
	2	3	第一节-第四节		
	2	4	第五节-第八节		
	3-4	1	第一节-第四节		
	3-4	2	第五节-第八节		
	3-4	4	第一节-第四节		

批次	周次	星期	节次	实验室	授课教师
E班	1	1	第七节-第八节	T2210	薛睿
	1	2	第三节-第四节		
	1	3	第五节-第八节		
	1	5	第一节-第四节		
	2-4	2	第一节-第四节		
	2-4	3	第五节-第八节		
	2-4	5	第五节-第八节		

批次	周次	星期	节次	实验室	授课教师
G班	1	1	第五节-第六节	T2612	薛睿
	1	2	第五节-第六节		
	1-4	4	第五节-第八节		
	1	5	第五节-第八节		
	2-4	1	第五节-第八节		
	2-4	2	第五节-第八节		

批次	周次	星期	节次	实验室	授课教师
F班	1	2	第七节-第八节	T2507	仇洁婷
	1	2	第九节-第十节		
	1-4	3	第五节-第八节		
	1	5	第一节-第四节		
	2-4	2	第一节-第四节		
	2-3	5	第五节-第八节		
	4	4	第一节-第四节		

批次	周次	星期	节次	实验室	授课教师
H班	1	2	第五节-第六节	T2506	仇洁婷
	1	3	第三节-第四节		
	1-3	4	第五节-第八节		
	1	5	第五节-第八节		
	2-4	1	第五节-第八节		
	2-4	2	第五节-第八节		
	4	3	第一节-第四节		

 致谢

- 特别鸣谢17级胡博涵、18级黎庚祉、19级罗尹同学在Trace测试框架上做出的贡献。
- 特别鸣谢19级罗翊洲同学在LA32R汇编工具上做出的贡献。

★开发板使用须知

1. 注意事项

★ 严禁将开发板带出实验室！

★ 禁止用手接触电路板上的芯片、金属线路等！ 人手有静电、汗液和灰尘，可能损坏开发板。

★ 使用开发板时，注意防水防尘，禁止在旁饮食！

★ 禁止随意拔出 开发板上的 跳线帽！



图1 装袋后，把袋口折起来

★ 禁止丢弃防静电袋！ 使用完毕时，将开发板正确装入防静电袋（如有），如图1所示。

★ 不要将USB线装入防静电袋！ 袋中只装开发板，USB线放袋外。

2. 开发板使用须知

在 T2507、T2210 上课的同学们，使用的是 EGO1开发板。

在 T2506、T2612 上课的同学们，使用的是 Minisys开发板。

请根据课表标注的实验室，点击查看对应开发板的使用说明。

EGO1使用说明

EGO1开发板的主芯片型号是XC7A35TCSG324-1，具有5200个逻辑Slice、50个36Kb的Block RAM存储器、90个DSP48E1数字信号处理器、5个时钟管理模块，内部时钟最高可达450MHz。

使用USB-TypeC线连接JTAG接口，打开电源开关即可使用。**插拔线材时，禁止使用蛮力。**

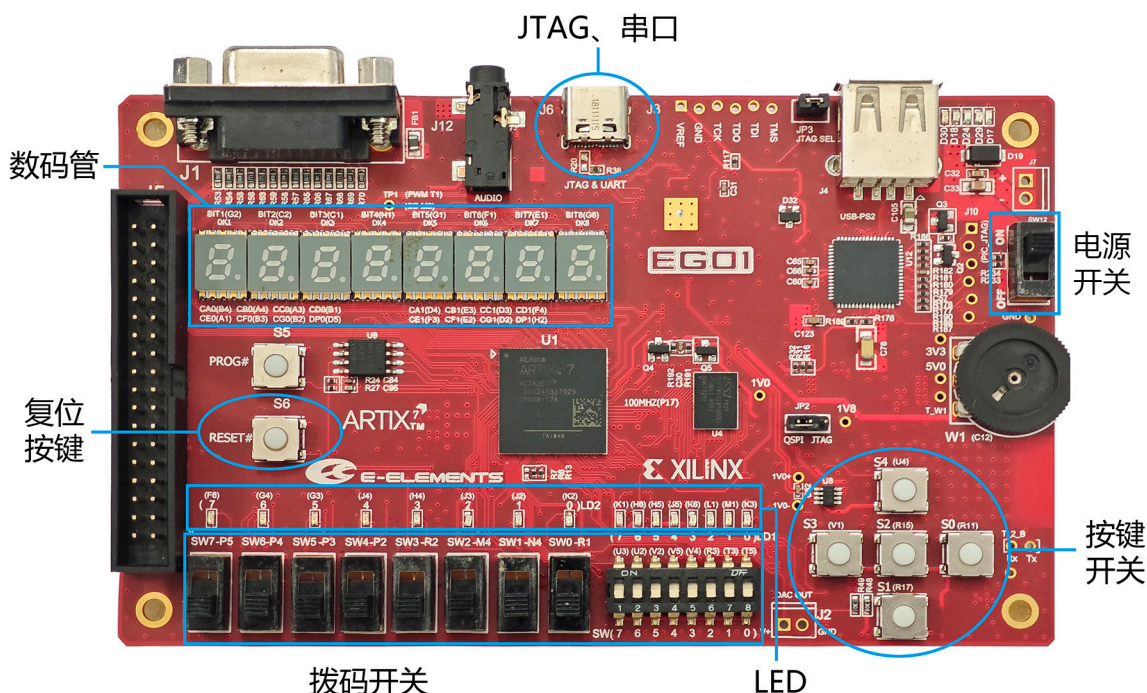


图2 EGO1开发板实物图

【时钟】：100MHz晶振时钟，连接到FPGA的 P17 引脚。

【复位】：S6按键开关，连接到FPGA的 P15 引脚，低电平复位。

【拨码开关】：8个拨码开关，及其相邻的8位数码开关，往上拨为高电平。

【按键开关】：S0~S4共5个通用按键开关，按下为高电平。

【数码管】：共阴极数码管，高4位是1组，低4位是另1组；段选、位选信号均为高电平有效。

【LED】：16位LED，高电平点亮。

上述基本外设对应的引脚约束文件：[ego1_pin.xdc](#)。

更多关于EGO1开发板的信息，请参考《[EGO1开发板用户手册](#)》。

Minisys使用说明

Minisys开发板的主芯片型号是**XC7A100TFGG484-1**，具有15850个逻辑Slice、135个36Kb的Block RAM存储器、240个DSP48E1数字信号处理器、6个时钟管理模块，内部时钟最高可达450MHz。

使用USB-TypeC线连接JTAG接口，打开电源开关即可使用。**插拔线材时，禁止使用蛮力。**

Minisys有2个版本，其中一个版本的 JTAG接口在电源开关旁边，请注意甄别。

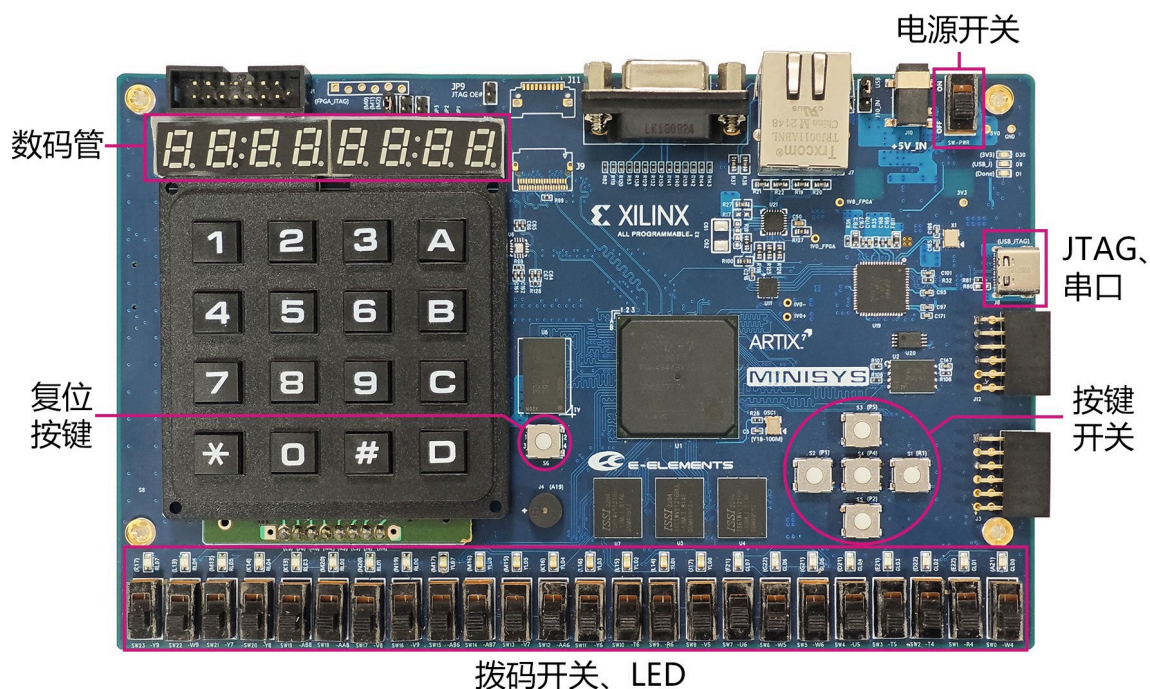


图3 Minisys开发板实物图

【时钟】：100MHz晶振时钟，连接到FPGA的 Y18 引脚。

【复位】：S6按键开关，连接到FPGA的 P20 引脚，高电平复位。

【拨码开关】：24个拨码开关，往上拨为高电平。

【按键开关】：S1~S5共5个通用按键开关，按下为高电平。

【数码管】：共阳极数码管，段选、位选信号均为低电平有效。

【LED】：24位LED，高电平点亮。

上述基本外设对应的引脚约束文件：[minisys_pin.xdc](#)。

更多关于Minisys开发板的信息，请参考《[Minisys开发板用户手册](#)》。

Verilog代码规范

1. 命名规范

1.1 文件命名规范

- 仿真文件应使用后缀 `_sim`，如 `modulename_sim`；
- 测试文件应使用后缀 `_tb`，如 `modulename_tb`。

1.2 模块命名规范

- 一个文件只定义一个 `module`；
- `module` 名应与文件名一致。

1.3 信号命名规范

- 用小写字母定义 `wire`、`reg` 和 `input`、`inout`、`output` 信号；
- 用大写字母定义 `parameter`、`localparam` 和宏定义；
- 信号名应反映信号的含义/用途，不要随意命名，更不能使用单个字母命名；
- 变量名若含有多个单词，用下划线分开，如 `ram_addr`，或使用驼峰命名法；
- 输入信号应使用后缀 `_i`，如 `addr_i`；
- 输出信号应使用后缀 `_o`，如 `data_o`；
- 时钟信号应使用前缀 `clk`，如 `clk_i`、`cpu_clk`；
- 复位信号应使用前缀 `rst`，如 `rstn_i`；
- 低电平有效的信号应带 `n` 后缀，如 `rstn_i`；
- 使用降序排列定义向量有效位顺序，最低位为0。

2. 代码编写规范

2.1 模块定义规范

- 每个模块加 ``timescale`，Vivado默认为 ``timescale 1ns / 1ps`；
- 出于方便考虑，推荐使用如以下代码所示的模块定义写法。


```

1  `timescale 1ns / 1ps
2
3  module some_module (
4      input wire      clk_i ,
5      input wire      rstn_i ,
6      input wire [1:0] sel_i ,      // 输入信号均为wire型
7      input wire [7:0] addr_i ,
8      output reg  [7:0] data_o      // 输出信号根据需要，可声明为reg或wire型
9  );
10
11      // 模块代码
12      .....
13
14  endmodule

```

2.2 参数规范

- 不需在模块实例化时设置的参数，应将其定义为局部参数（`localparam`）；
- 全局参数建议放在I/O端口前面，如图3所示。

```

1  module some_module #(
2      parameter PARAM1 = 8,    // 参数默认值
3      parameter PARAM2 = 2
4  )(
5      input wire      clk_i ,
6      input wire      rstn_i,
7      .....
8  );

```

2.3 模块实例化规范

- 模块实例应用 `U_xx_x` 表示（多次例化用序号0、1、2等表示）；
- 实例化模块时，推荐采用以下写法。

```

1  some_module #(
2      .PARAM1    (`WIDTH),
3      .PARAM2    (`DEPTH)    // 参数
4  ) U_some_module_0 (        // 实例名
5      .clk_i      (clk_fpga ),
6      .rstn_i     (rst_fpga_n),
7      .....
8  );

```

2.4 通用规范

- 尽量采用参数化设计；

- 所有的 `if` 语句应有与之对应的 `else`，特别是组合逻辑；
- `case` 语句应考虑 `default` 情况，特别是组合逻辑；
- `if-else` 语句尽量不要嵌套太多；
- 在RTL级代码中不能含有 `initial` 结构，也不可对任何信号进行初始化赋值。若需要初始化，应采用复位的方式：

```

1 // 错误的初始化
2 initial begin
3     data1    = 8'h12;
4     signal2 = 12'hABC;
5 end
6
7 // 错误的初始化
8 reg [ 7:0] data1    = 8'h12;
9 reg [11:0] signal2 = 12'hABC;
10
11 // 正确的初始化
12 always @ (posedge clk or posedge rst) begin
13     if (rst) begin
14         data1    <= 8'h12;
15         signal2 <= 12'hABC;
16     end else begin
17         .....
18     end
19 end

```

- 尽量不产生未连接的端口；
- 信号赋值或实例化时，应保证数据位宽匹配；
- 顶层模块的输出信号必须被寄存；
- 常量应标注其位宽，如 `1'b0`；
- 不建议使用 `for`、`repeat`、`while` 等语句；
- 如非必要，不使用 `integer` 类型；
- 尽量不使用复杂的表达式，可以使用三目运算符 `?:`。

2.5 组合逻辑规范

- 如果需要使用 `always` 语句实现组合逻辑，则敏感信号列表应为 `*`，即：

```

1 always @(*) begin
2     .....
3 end

```

- 组合逻辑 `always` 块使用阻塞赋值 `=`。

2.6 时序逻辑规范

- 采用同步设计，避免使用异步逻辑（全局信号复位除外）；
- 同步时序逻辑的 `always` 块中有且只有一个时钟信号，并且在同一个沿动作（如上升沿），一般不使用下降沿；
- 在时序 `always` 块的敏感信号列表中必须都是边沿触发，不允许出现电平触发；
- 敏感信号列表中不允许出现表达式；
- 除异步复位之外，敏感信号列表中不允许同时出现 `posedge` 和 `negedge`；
- 时序逻辑语句块中统一使用非阻塞型赋值 `<=`；
- 一个 `always` 块只对一个变量赋值，或只对有关系的一组信号赋值；
- 建议多使用中间变量。

★常见问题汇总

本页面汇总了历届同学在本课程中可能会出现的问题及其解决办法，仅供参考。



特别说明

以下罗列的问题并无特别的顺序，建议同学们先大体浏览各个问题。在实验中遇到问题时，先寻找类似的问题看看能否找到解决方法。

1. CPU设计

1.1 数据通路图用什么软件画？

可以使用Visio、Visual Paradigm、ioDraw、draw.io、ProcessOn等

1.2 定义了宏，但报错显示未定义？

在每一个需要使用宏定义的.v文件内添加`include "defines.vh" 语句。此外，引用宏定义时需在前面加上`符号。例如，`define MACRO1 111，则引用时应该是`MACRO1。

1.3 移位运算的移位位数超过`d32，或者是负数，怎么办？

把移位位数看成是无符号数，截取最低5位才是有效的移位位数。

1.4 load指令从DRAM取数据时，取到的是0

ALU计算得到的是字节地址，DRAM的地址是字地址，需排查接线是否有误。此外，也可能是前面的store指令往该存储单元写了0的数据。

1.5 出现跟时钟有关的Critical Warning

检查是否同时使用了时钟信号的上升沿和下降沿。

1.6 AXI Trace测试通过，但下板失败

Trace框架只会测试SoC的主存访问，不会测试外设访问。如果Trace测试通过，但下板测试不通过，首先应检查DCache的Uncached访问、SoC的外设访问是否存在问题。调试时，可通过把C_TEST程序的.coe文件导入主存然后进行功能仿真或下板抓取波形进行分析。

1.7 如何提高CPU主频？

优化措施需要针对瓶颈展开，才能事半功倍。从理论上分析，单周期CPU的频率取决于执行最耗时的指令，流水线CPU的频率取决于最耗时的流水线阶段。因此，想要提高频率，需要找出

数据通路中最耗时的部分。

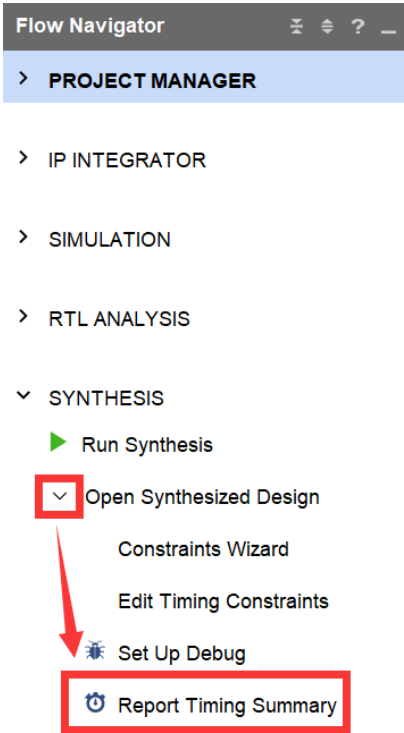
在实践中，可借助Vivado综合后生成的时序报告来找出关键路径，然后通过逻辑优化或切割等方法来降低关键路径的延迟，详情可参考《[FPGA时序分析之关键路径-CSDN Blog](#)》。

使用Vivado查看关键路径的具体操作步骤如下：

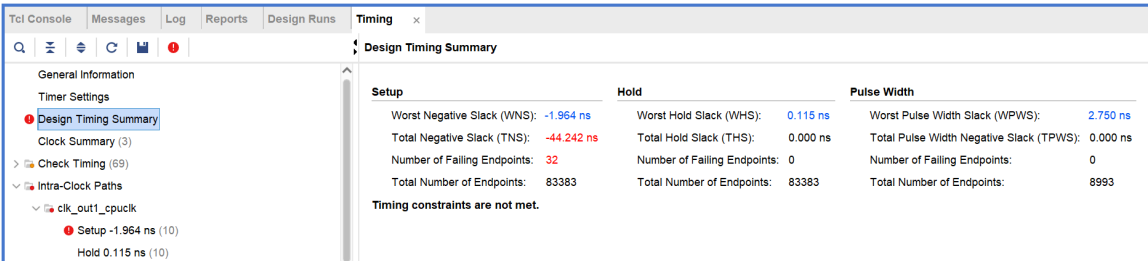
Step1: 更改时钟IP核设置（或修改分频器参数），提高CPU主时钟的频率至期望值；

Step2: 通过快捷键 `F11` 或GUI操作，对工程进行综合（Synthesis）；

Step3: 综合完成后，在Vivado默认界面左侧的 `Flow Navigator` 下，找到并展开 `SYNTHESIS` 下的 `Open Synthesized Design`，点击 `Report Timing Summary`，如下图所示。



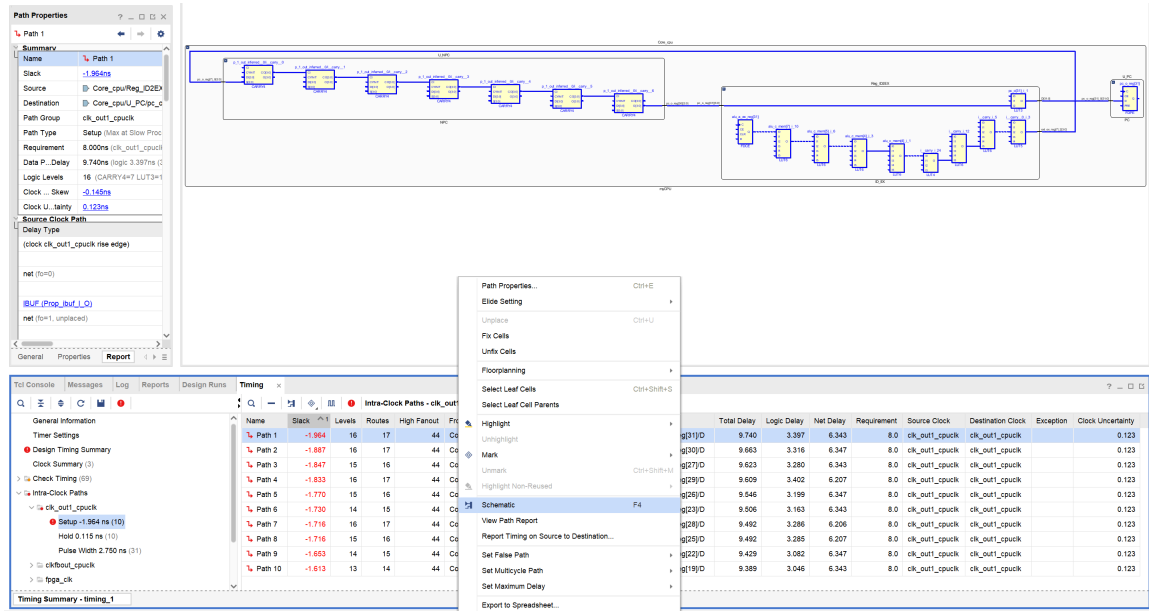
随后将弹出 `Report Timing Summary` 的对话框，直接点击 `OK` 按钮，即可在Vivado默认界面下方的区域查看时序总结报告。工程中的时序违例相关信息将被红色字体标注，如下图所示。



Step4: 展开标红的子项，查看关键路径，如下图所示。

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	-1.964	16	17	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg31]D	9.740	3.397	6.343	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 2	-1.887	16	17	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg30]D	9.663	3.316	6.347	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 3	-1.847	15	16	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg27]D	9.623	3.280	6.343	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 4	-1.833	16	17	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg29]D	9.609	3.402	6.207	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 5	-1.770	15	16	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg28]D	9.546	3.199	6.347	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 6	-1.730	14	15	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg29]D	9.506	3.163	6.343	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 7	-1.716	16	17	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg28]D	9.492	3.296	6.206	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 8	-1.716	15	16	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg29]D	9.492	3.285	6.207	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 9	-1.653	14	15	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg29]D	9.429	3.082	6.347	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123
Path 10	-1.613	13	14	44	Core_cpuReg_ID2EX[alu_a_reg31]C	Core_cpuU_PC[pc_o_reg19]D	9.389	3.046	6.343	8.0	clk_out1_cpuclock	clk_out1_cpuclock		0.123

若想借助电路图分析，可在某条关键路径上打开右键菜单并点击 Schematic，如下图所示。



补充说明

Step1并非必须 —— 当CPU主频较低时，工程中不存在时序违例，此时Step3的时序报告仍会显示关键路径，但并非以红色字显示。

1.8 只做完单周期能不能及格？

- 如果是2人小组，则需要：
 - 单周期CPU实现所有指令，能够跑通Basic Trace；
 - 完成理想流水线CPU，通过功能仿真，并在课上通过现场检查；
 - 为单周期CPU实现AXI总线，跑通lw.dump、sw.dump的功能仿真测试。

完成以上内容，并且还要认真完成报告，才能及格。

- 如果实在找不到人小组合作（比如重修的同学），则需要：
 - 完成A组或B组指令的单周期CPU，跑通Basic Trace；
 - 完成理想流水线CPU，通过功能仿真，并在课上通过现场检查；

(3) 完成[实验2的I/O接口编程任务](#)，并在课上通过现场检查；

完成以上内容，并且还要认真完成报告，才能及格。

2. Trace验证

2.1 虚拟机启动不了，一直转圈

重启电脑后，3分钟内启动虚拟机。或更换座位。

2.2 MobaXTerm连接时一直报错 `Network error, connect refused`

建议换用实验室的WSL2虚拟机环境。

关闭防火墙和杀毒软件，重启电脑。

2.3 VSCode连接远程Trace平台，报错 `Bad owner or permissions on .ssh/config`

建议换用实验室的WSL2虚拟机环境。

解决方法可参考《[Win10下Bad owner or permissions on .ssh/config的解决办法-腾讯云](#)》。

2.4 编译报错提示“multiple target patterns”

检查mySoC目录下是否有Zone.Identifier文件，将其全部删除后重试。

2.5 build的时候报错 `Timescale missing on this module`

在相应的 .v文件 里加上 `/* verilator lint_off TIMESCALEMOD */` 或删除所有文件里的 ``timescale 1ns / 1ps` 语句。

2.6 明明在虚拟机里修改过代码了，但波形还是原来的波形

尝试在 `make` 前先执行 `make clean` 命令。

2.7 Trace验证时，所有指令都显示Time out

仔细按照[Trace测试说明](#)逐一检查debug信号、模块名、存储器参数设置等是否符合要求。

此外Time out可能是因为CPU本身陷入了长时间等待状态，比如发出了取指请求，但一直没有收到指令；或者一条指令执行完之后，一直没有取下一条指令；又或者数据访存出问题等等。此种情况，可结合Trace测试的仿真波形进行分析。

2.8 复位信号到底是低电平还是高电平？

Trace验证时提供的复位信号是高电平复位。EGO1开发板的复位按钮是低电平复位，Minisys开发板则是高电平复位。

2.9 Trace验证时，REFERENCE 的PC和 MYCPU 的PC不一致

检查复位逻辑，务必保证复位后取第一条指令的PC为 `32'h0`。

对于流水线CPU，确保一条指令比对完后、下一条指令比对前，拉低 `debug_wb_have_inst`。

检查程序是否出现了错误的跳转，或检查PC的更新逻辑是否有bug。

2.10 输入命令查看波形，结果没反应？

不要使用root用户。

2.11 Trace测试用例的汇编代码在哪里可以找到？

看Trace测试包下的 `asm` 目录，即 `cdp-tests / asm`，或者根据指令集点击查看：[miniRV](#)、[miniLA](#)。

2.12 仿真Trace通过所有指令，但下板卡在某一指令的测例

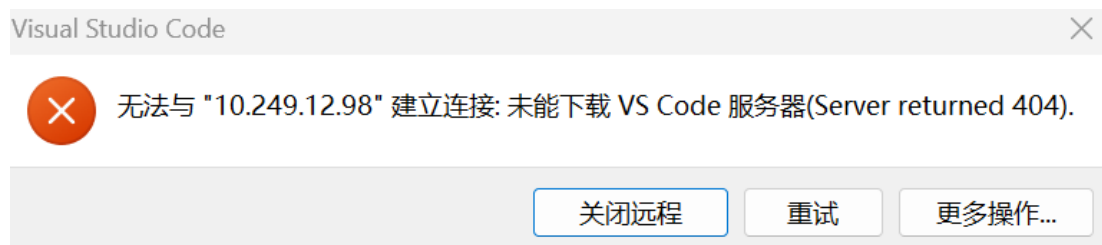
检查CPU和主存（`IROM`、`DRAM` 或 `bram_axi`）是否使用不同的时钟。

2.13 怎么保证提交代码后Trace自动评测一定能过

只要你自己能在虚拟机里能够通过Trace测试，就把mySoC目录压缩提交就行。就算评测真的有问题，也会有人工复核，必要时会通过课程群联系你。

2.14 VSCode连接Trace远程平台问题

建议换用实验室的WSL2虚拟机环境。



推荐用MobaXTerm来连接Trace远程平台，而不是用VSCode连接 —— VSCode连接远程服务器会占用较多网络资源，尤其是本课程有很多同学同时使用远程平台！

2.15 执行make提示“No rule to make”

```
make: *** No rule to make target 'mySoC/myCPU', needed by 'build'. Stop.
```

执行make前，先确保已经通过 `cd` 命令进入了 `cdp-tests` 目录。如果目录没问题，则检查 `cdp-tests` 目录是否包含 `Makefile` 文件。如果包含，但仍然提示“No rule to make”，则备份好个人代码，删除现有 `cdp-tests` 目录并重新下载和编译。

2.16 我的Cache和乘法器已经通过测试，为什么Trace测试过不了

如果ICache、DCache、乘法器模块都能跑通计算机组成原理实验的测试，但是在Trace测试时不通过，是正常现象，毕竟不同测试的严格程度不一样。与其怀疑测试框架有问题，不如根

据波形细心完成调试。

3. 下板

3.1 开发板Auto Connect失败，是开发板坏了吗？

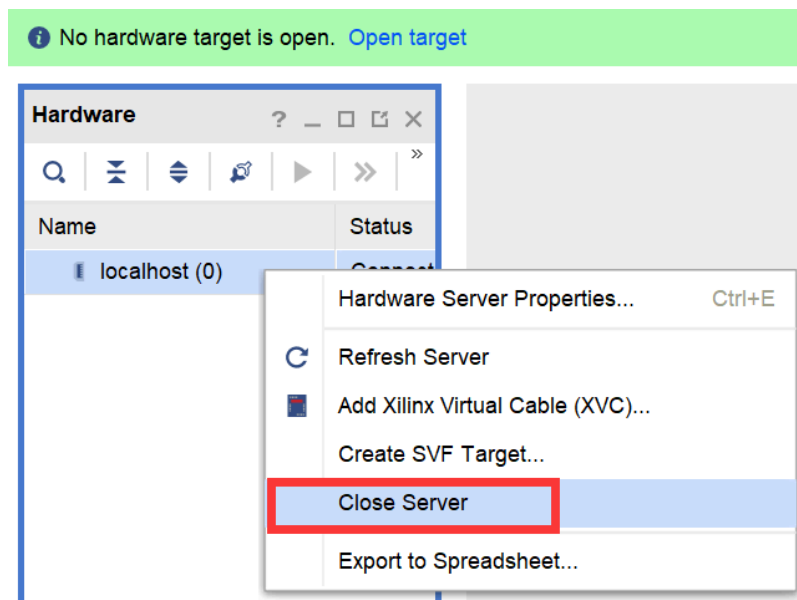
确保USB数据线正确连接开发板的JTAG接口 —— 对于Micro USB接口的Minisys开发板，JTAG接口在数码管右上方。

确保电源连接无误 —— 对于Micro USB接口的Minisys开发板，需额外连接5V的电源适配器；对于USB TypeC接口的Minisys开发板，使用USB数据线供电即可。

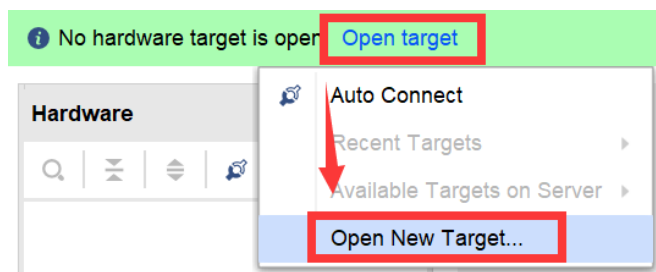
确保开发板右上角的电源拨码开关处于 ON 的状态。

排除以上硬件连接问题后，若Vivado中Auto Connect仍然失败，则按以下步骤操作：

Step1: 关闭 Hardware 窗口的 server，如下图所示。

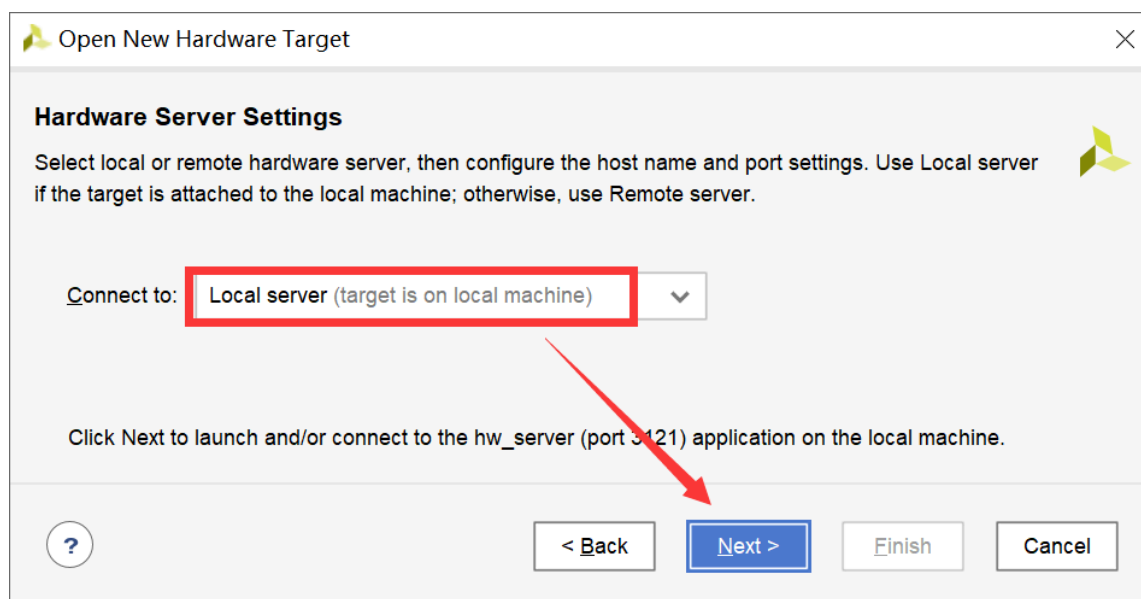


Step2: 点击 Open target -> Open New Target...，如下图所示。

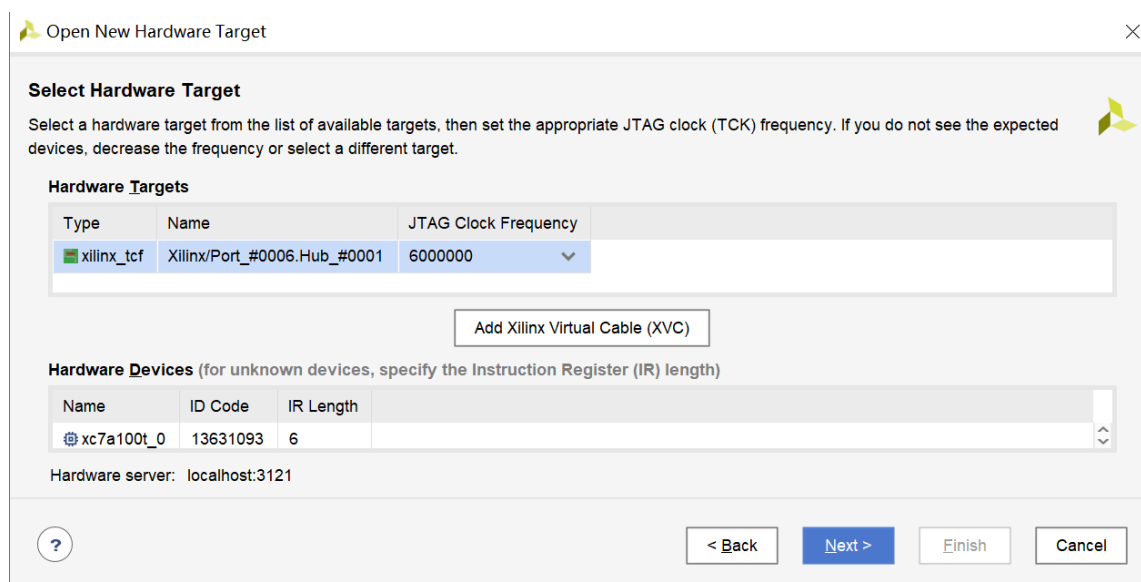


随后将弹出一个窗口，点击 Next 按钮。

Step3: 确保 Connect to: 选择的是 Local server，并继续点击 Next 按钮，如下图所示。



等待进度条走完，将显示当前连接的 Hardware Targets，如下图所示。



实际操作中，若上图所示的界面未显示有 Hardware Targets，则点击 Back 按钮，并重新尝试Step3 —— Micro USB接口的开发板版本较老，有时需多重复尝试几次才能连上。

若重复尝试Step3若干次，仍然连接失败，则先更换数据线，再从Step2开始重新尝试；若更换数据线后仍然连接失败，则换一块开发板再尝试。若更换开发板仍然连不上，可能是电脑尚未安装驱动。打开 C:\Xilinx\Vivado\2018.3\data\xicom\cable_drivers\nt64\digilent 目录，双击 install_digilent.exe 安装驱动程序，再重新尝试连接。若仍然连接不上，则将该开发板交给任课老师并说明相关情况。

Step4: 继续点击 Next 按钮，并点击 Finish 按钮，即可连接开发板。

3.2 综合/实现/比特流跑了半天跑不出来，怎么办？

首先代码越复杂，IROM里的程序越大，综合/实现/比特流是会越慢的，甚至可能需要几十分钟，请耐心等待。

检查工程名、所在路径是否有中文或空格。

尝试先关闭工程，找到工程所在目录，将除 `project_name.xpr` 和 `project_name.srcs` 之外的文件和文件夹删掉，再尝试生成比特流。

也有可能是代码有bug，导致Vivado出bug，从而跑不出来。这种情况下，需要检查代码中不规范不合理的地方，一一改过来。

3.3 综合/实现/比特流报错 `Combinational Loop Alert`

设计中存在组合逻辑环路，需要根据报错信息提示的信号/模块，找出这个环并解决之。

3.4 跑实现的时候跑了很久，底下显示有红色数字的负数

检查CPU和DRAM是否使用不同的时钟。

3.5 仿真和下板不一致，怎么办？

仿真和下板出现不一致的情况，可能是以下几个原因：

- (1) **代码规范性**问题，比如寄存器未赋初值、阻塞赋值和非阻塞赋值混用、多驱动等；
- (2) 关键路径延迟太长，导致不满足 `Setup time` 或 `Hold time` —— 可尝试优化关键路径或降低CPU主频；
- (3) 组合逻辑出现竞争或冒险。

常见的问题包括：阻塞赋值和非阻塞赋值混用、组合逻辑的 `if` 语句没有 `else`、`case` 语句没有 `default`、时钟频率不合适，或下板用的汇编程序有bug等等。

仿真和下板不一致时，问题可能比较隐晦，需要耐心排查代码。查错时，也可以参考Vivado的Warning信息、log日志等；当然也可使用[FPGA在线调试方法](#)抓取运行时波形进行分析。

3.6 `Timing Report` 中的 `WNS`、`TNS` 等几个时间都显示 `NA`

IP核的分频器不要设置成Global buffer。如果是自己实现分频器，不能有复位信号。

检查时钟信号是否使用有误，比如是否同时使用 `fpga_clk` 和 `cpu_clk` 来驱动除了时钟模块以外的多个模块。